Plano de Estudos — Backend .NET + SQL Server (v2)

Oct 15, 2026 · @Mari

Visão geral

Em 24 semanas, com 3 horas por semana (72 horas no total), construir um sistema pequeno, seguro e demonstrável de gestão para uma igreja. O projeto começa na semana 1 e cresce junto com cada tema estudado.

Duração	Disponibilidade	Projeto prático
24 semanas (6 meses)	3 h/semana, em 3 sessões de 1 h	Cadastro de pessoas, presença e aniversariantes para uma igreja, já preparado para várias igrejas

O que mudou em relação à v1

- O sistema começa na semana 1, e não na 19. Todo exercício de SQL roda no banco da igreja.
- O isolamento entre igrejas (`OrganizacaoId`) entra no modelo desde a primeira tabela, e não só na semana 22.
- Foram incluídos autenticação, perfis e uma interface em Razor Pages/MVC, que estavam ausentes.
- TCP/UDP e parsing binário saíram deste projeto. Esses temas ficam para as demandas reais do trabalho.
- RabbitMQ e Redis viraram extensões opcionais, feitas só depois que o fluxo principal funcionar.
- Há entregáveis pequenos por semana e um MVP dimensionado para 72 horas.
- Foram incluídos auditoria (técnica e de acesso), backup/restauração e deploy.

Produto e MVP

O alvo destes 6 meses é **uma igreja**: a sua. Atender igrejas do Brasil inteiro é a visão de futuro, a ser buscada só depois de validar o sistema com as pessoas que vão usá-lo, como a secretaria e os líderes.

Funcionalidades da visão completa: cadastro centralizado de pessoas, lista de presença diária, aniversariantes, contatos e endereços conforme a permissão de cada usuário, relatórios para viagens e eventos, e administração de igrejas, unidades e usuários.

MVP que cabe em 72 horas

Prioridade	Funcionalidade	Critério de conclusão
1 — Essencial	Login e perfis (Admin, Secretaria, Líder)	Nenhuma tela ou endpoint funciona sem login; cada perfil vê só o que pode
2 — Essencial	Cadastro de pessoas	Cadastrar, editar, pesquisar com paginação e validar os campos necessários
3 — Essencial	Atividades e presença	Criar uma atividade e registrar presença, sem permitir duplicidade de pessoa na mesma atividade
4 — Essencial	Aniversariantes	Filtrar por período, exibindo só os dados permitidos ao perfil
5 — Essencial	Isolamento entre igrejas	Teste automatizado que tenta ler dados de outra organização e falha
6 — Importante	Relatório de viagem	Lista com campos mínimos, acesso restrito e exportação registrada na auditoria
7 — Opcional	Fila de parabéns (RabbitMQ), cache (Redis), Docker, CI/CD	Só depois que os itens 1 a 5 estiverem estáveis

Fora do MVP, de propósito: dados de menores (ministério infantil), documentos (CPF/RG), dados de saúde, recuperação de senha por e-mail (no início, o admin redefine a senha) e cadastro self-service de novas igrejas.

Arquitetura e isolamento entre igrejas

O isolamento é uma regra de segurança, e não um filtro de pesquisa. Toda tabela de dados tem `OrganizacaoId` desde o primeiro script, e toda operação valida o contexto da organização no backend, inclusive buscas por ID e exportações.

Conceito	Responsabilidade
Organização	A igreja/instituição. É o limite principal de isolamento dos dados
Unidade	Congregação ou filial dentro de uma organização (opcional no MVP)
Pessoa	Cadastro centralizado no escopo da organização
Usuário	Conta de acesso com um perfil. Pode ou não corresponder a uma Pessoa
Atividade	Culto, reunião, evento ou viagem
Presença	Pessoa × Atividade, com chave única `(AtividadeId, PessoaId)`
Auditoria	Eventos de alteração e de acesso, com leitura restrita

Por que só ter `OrganizacaoId` não basta: a coluna existe, mas alguém precisa aplicá-la. A organização vem do usuário autenticado (claim do login), nunca de um parâmetro enviado pelo cliente.

Ordem de estudo das técnicas de isolamento

1. **Filtros explícitos e centralizados na aplicação:** um serviço `IContextoOrganizacao` usado por todos os repositórios. É por onde começar.
2. **Global Query Filters do EF Core:** o filtro é aplicado automaticamente em toda consulta da entidade. Estude também quando `IgnoreQueryFilters()` vira um risco.
3. **Row-Level Security (RLS) no SQL Server:** uma camada extra no banco. Exige configurar o contexto da sessão (`SESSION_CONTEXT`) em cada conexão, com cuidado com o pool de conexões.

As três não se excluem. O objetivo é implementar a 1, testar a 2 e a 3 e documentar os trade-offs.

Camadas da solução: Domain (entidades e regras), Application (casos de uso), Infrastructure (SQL Server, EF Core) e Web (Razor Pages/MVC + API). Sem abstrações que ainda não têm uso.

Segurança, LGPD e auditoria

Regra de ouro: **dados fictícios do início ao fim do estudo**. Dados reais da igreja só entram depois que login, autorização por perfil e isolamento entre organizações estiverem implementados e testados.

LGPD

- Nome, telefone, endereço e nascimento são dados pessoais. Pertencer a uma igreja revela convicção religiosa, que é **dado pessoal sensível** (LGPD, art. 11).
- Antes de usar dados reais, defina finalidade, base legal, transparência com os membros, tempo de retenção, quem acessa e como atender os pedidos dos titulares (acesso, correção, exclusão). Busque orientação adequada para a igreja.
- Guarde só o necessário. Menores, documentos e saúde ficam fora do MVP.

Dois tipos de auditoria

Tipo	O que registra	Exemplos
Auditoria de alteração	Quem criou, alterou ou excluiu, quando e o que mudou	Pessoa editada, presença removida, perfil alterado
Auditoria de acesso	Quem consultou ou exportou dados pessoais, e com qual filtro	Relatório de viagem exportado, busca de endereços

Os registros de auditoria têm acesso restrito (só Admin) e não podem ser editados pela aplicação.

Logs técnicos

- Registre IDs e códigos de erro, e nunca nome, telefone ou endereço.
- Mensagens de erro para o usuário não expõem detalhes internos.

Boas práticas gerais: senhas com hash (ASP.NET Identity), HTTPS, segredos fora do código (user-secrets/variáveis de ambiente), exportações mínimas e restritas. Não copie código, dados ou informações internas da empresa para este projeto.

Rotina semanal — 3 horas

Cada semana termina com uma evidência concreta no repositório: um commit, um script, um teste ou uma nota curta no README.

Sessão	Tempo	Atividade	Resultado
Dia 1 — Aprender	60 min	Documentação e conceito, com uma anotação curta	Entender um conceito central
Dia 2 — Praticar	60 min	Aplicar no sistema da igreja	Código, consulta ou teste funcionando
Dia 3 — Entregar	60 min	Finalizar, medir, commitar e registrar o que aprendeu	Entregável da semana no Git

Distribuição aproximada do esforço: 20% teoria, 50% implementação, 20% documentação e 10% revisão. **Se a semana apertar, reduza o escopo em vez de acumular tarefas.** Uma semana perdida empurra o calendário, não dobra a semana seguinte.

Calendário de 24 semanas

Cada mês termina com algo funcionando. O banco nasce na semana 1, a API no mês 3, as telas com login no mês 4, e o sistema é demonstrado na semana 24.



roteiro de 6 meses · 1 portão

Os meses 1 a 4 constroem a base (banco, API, login e isolamento). O portão entre os meses 4 e 5 é a regra: nenhum dado real entra antes dele.

Mês 1 — Banco da igreja e fundamentos de SQL

Semana	Foco	Entregável
01	Modelo inicial	Tabelas `Organizacao`, `Pessoa` e `Atividade` com PK/FK e `OrganizacaoId`; script de criação versionado no Git; cerca de 20 registros fictícios; README inicial
02	Presença e integridade	Tabela `Presenca` com chave única `(AtividadeId, PessoaId)`, FKs e CHECKs; consultas básicas validadas com os dados pequenos
03	Massa de dados e planos de execução	Script gerador (100 mil pessoas, 1 milhão de presenças); plano de uma consulta de pessoas analisado (scan/seek, estimativas, operadores caros)
04	Índices e métricas	Índice composto com `OrganizacaoId` à esquerda e `INCLUDE`; comparação antes/depois com `STATISTICS IO/TIME`; início do exercício de aniversariantes

Mês 2 — SQL avançado e integridade

Semana	Foco	Entregável
05	Joins e cardinalidade	Consulta de histórico de presença revisada (conversões implícitas, estimativas), com resultado comprovadamente equivalente
06	Consultas avançadas	Frequência por pessoa com `ROW_NUMBER` / `PARTITION BY`, CTE, `CROSS APPLY` (últimas 3 presenças) e paginação com `OFFSET/FETCH`
07	Transações e concorrência	Procedure de registrar presença com `TRY/CATCH`, `THROW` e `XACT_ABORT`; duplicidade simulada em duas sessões; nota sobre isolamento e `NOLOCK`
08	Diagnóstico e isolamento no banco	Query Store e DMVs (relatório com sintoma, hipótese, evidência e próximo teste); prova de conceito de RLS num banco de testes

Mês 3 — C# e API real

Semana	Foco	Entregável
09	C# moderno	Entidades com regras de domínio (nullable, validações, LINQ) e tratamento de exceções revisado
10	Camadas e injeção de dependência	Solução Domain/Application/Infrastructure/Web; ciclos de vida justificados; `IContextoOrganizacao`
11	EF Core e async/await	`DbContext` mapeando o banco; repositório assíncrono de Pessoa com `CancellationToken`; decisão documentada entre migrations e scripts
12	API ASP.NET Core	CRUD de pessoas com validação, paginação, erros padronizados (`ProblemDetails`) e Swagger

Mês 4 — Login, permissões e interface

Semana	Foco	Entregável
13	Autenticação	ASP.NET Identity com cookie; usuário vinculado a uma organização; login/logout; admin inicial criado por seed
14	Autorização	Perfis Admin/Secretaria/Líder com policies; `OrganizacaoId` lido da claim; nenhum endpoint aberto
15	Interface	Telas em Razor Pages/MVC: lista paginada e cadastro de pessoas
16	Testes e isolamento	xUnit para as regras; teste de integração em que o usuário da igreja A tenta ler, editar e exportar dados da B e recebe 404/403; comparação com Global Query Filter

Mês 5 — Funcionalidades de uso diário

Semana	Foco	Entregável
17	Atividades	Cadastro e listagem de cultos, reuniões e eventos
18	Lista de presença	Tela para marcar presença rapidamente, sem duplicidade, com histórico por pessoa
19	Aniversariantes	Tela com filtro por período usando a consulta otimizada do mês 1, respeitando o perfil
20	Pesquisa e relatório de viagem	Busca paginada por nome/telefone; relatório com campos mínimos e exportação restrita; plano de execução conferido

Mês 6 — Segurança, operação e demonstração

Semana	Foco	Entregável
21	Auditoria	Auditoria de alteração e de acesso (incluindo exportações), com tela só para Admin; logs revisados sem dados pessoais
22	Backup e deploy	Backup restaurado em outro banco e conferido; deploy com dados fictícios; segredos fora do código
23	Documentação	README (como rodar, variáveis, decisões técnicas, limitações, checklist de LGPD); Docker se couber
24	Demonstração e próximos passos	Demo para alguém da igreja; feedback registrado; revisão sem anotações; plano dos próximos 90 dias

Exercício destaque: consulta de aniversariantes

Este exercício junta modelagem, SQL Server e desempenho. Ele começa na semana 4, com a massa grande de dados, e vira a tela da semana 19.

Ponto de partida. Aplicar funções sobre a coluna impede o uso eficiente de um índice em `DataNascimento`, e o resultado tende a ser um scan:

```
SELECT PessoaId, Nome, DataNascimento
FROM Pessoa
WHERE OrganizacaoId = @OrganizacaoId
  AND MONTH(DataNascimento) = @Mes
  AND DAY(DataNascimento) = @Dia;
```

Alternativa a medir. Uma coluna calculada persistida com mês e dia, indexada junto com a organização:

```
ALTER TABLE Pessoa
  ADD MesDiaNascimento AS (MONTH(DataNascimento) * 100 + DAY(DataNascimento))
PERSISTED;

CREATE INDEX IX_Pessoa_Org_MesDia
  ON Pessoa (OrganizacaoId, MesDiaNascimento)
  INCLUDE (Nome, DataNascimento);

-- período de 10/03 a 25/03
WHERE OrganizacaoId = @OrganizacaoId
  AND MesDiaNascimento BETWEEN 310 AND 325;
```

O que medir e registrar

- O plano de execução de cada versão (scan ou seek) e as estimativas contra as linhas reais.
- Os logical reads e o tempo (`SET STATISTICS IO, TIME ON`).
- O custo do índice na escrita e no espaço.
- Casos de borda: um período que vira o ano (20/12 a 10/01) e quem nasceu em 29/02.

O melhor desenho depende da consulta, do volume e da distribuição dos dados. Por isso, medir faz parte do exercício, e a conclusão vem dos números.

Extensões opcionais e fora do escopo

Estas extensões só entram quando o fluxo principal (login, pessoas, presença, aniversariantes e isolamento) estiver estável e testado. Podem ocupar semanas de folga ou o próximo ciclo de 90 dias.

Extensão	Caso real no sistema	O que ensina
RabbitMQ	Fila de mensagens de parabéns aos aniversariantes (e-mail/WhatsApp)	Exchange, queue, ack, retries, dead-letter e idempotência (não mandar a mesma mensagem duas vezes)

Extensão	Caso real no sistema	O que ensina
Redis	Cache de uma consulta muito frequente, só se a medição mostrar ganho	TTL, invalidação e serialização
Docker	Subir a aplicação e o SQL Server com um comando	Imagens, compose e variáveis de ambiente
CI/CD	Build e testes automáticos a cada push	Pipeline e qualidade contínua

Fora deste projeto: TCP/UDP e parsing binário (bytes, endianness, CRC). São úteis no trabalho com protocolos de rastreadores, mas a melhor forma de aprendê-los é com demandas reais do dia a dia, e não no sistema da igreja.

Checklist, avaliação e referências

Checklist de cada mês

- [ ] Consigo explicar os conceitos sem ler minhas anotações.
- [ ] Implementei a funcionalidade no sistema da igreja.
- [ ] Consigo justificar as decisões técnicas e os riscos delas.
- [ ] Registrei evidências no Git (código, medições, README).
- [ ] Sei quais lacunas devo estudar em seguida.

Ao final das 24 semanas, devo conseguir:

Área	Evidência esperada
SQL Server	Analisar um plano de execução, justificar índices com métricas e investigar gargalos
.NET	Construir uma aplicação organizada, assíncrona e testável, com API e telas
Arquitetura	Explicar a responsabilidade de cada camada e as decisões tomadas
Segurança	Demonstrar login, perfis e isolamento entre organizações com testes
Operação	Mostrar um backup restaurado, o deploy e a auditoria funcionando

Área	Evidência esperada
Autonomia	Investigar um problema, formular hipóteses, validar e documentar

Referências

- SQL Server: Monitorar e ajustar desempenho, Transações e bloqueios, Row-Level Security
- .NET: Documentação C#, ASP.NET Core, EF Core Global Query Filters, ASP.NET Core Identity
- Extensões: RabbitMQ Tutorials, Redis com .NET
- LGPD: Lei 13.709/2018

Este é um plano de aprendizagem, e não uma especificação pronta para produção. Antes de usar o sistema com dados reais de uma igreja, revise arquitetura, segurança, LGPD, backups e operação com orientação adequada.